SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation
Weightage: 20%

QUESTIONS: **Total Marks: 20**

1. **Servlet Architecture & Workflow**

Question:

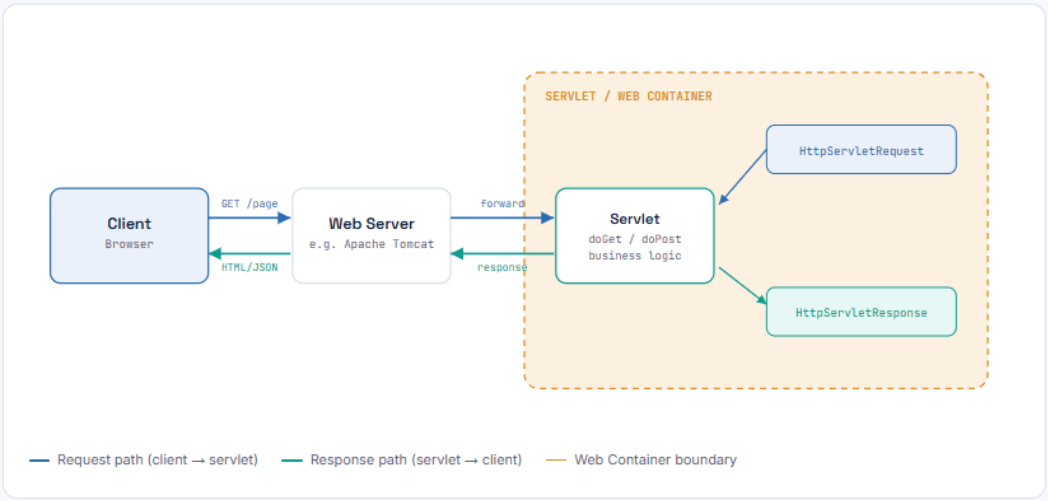
Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

The Big Picture

A servlet never talks to the browser directly. The Web Container sits in between — it owns the servlet's entire lifecycle and does the networking work for it.



What the Web Container Actually Does

The container (e.g. Tomcat's Catalina engine) is the runtime that manages servlets so you never write networking or thread-handling code yourself.

`lifecycle`

Loads & instantiates the servlet

Loads the servlet class and creates a single instance the first time it's needed, then calls `init()` once.

`routing`

URL mapping

Matches the incoming request's URL against `web.xml` or `@WebServlet` mappings to pick the right servlet.

`concurrency`

Multithreading

Spawns a new thread per request and calls `service()` on the shared servlet instance — one object, many threads.

`objects`

Builds request & response objects

Creates a fresh `HttpServletRequest` / `HttpServletResponse` pair for every single request.

`security`

Session & security management

Handles cookies, `HttpSession` tracking, and declarative security constraints.

`cleanup`

Destroys the servlet

Calls `destroy()` once when the app is shutting down or being redeployed.

The Complete Request–Response Journey

Eight steps, from the moment a link is clicked to the moment the page renders.

1. **Client sends an HTTP request**
   The browser sends a `GET` or `POST` request to a URL, e.g. `/products`.

2. **Web server receives it**
   The server checks the URL. If it maps to a servlet, the request is handed off to the Web Container.

3. **Container locates or loads the servlet**
   If this is the first request ever for that servlet, the container loads the class and calls `init()` exactly once.

4. **A new thread is created**
   The container spins up a fresh thread for this request so multiple users can be served at once.

5. **Request & Response objects are built**
   The container creates a brand-new `HttpServletRequest` and `HttpServletResponse` pair for this call only.

6. **`service()` dispatches to your code**
   Based on the HTTP method, it routes to `doGet()`, `doPost()`, etc., where your business logic runs.

7. **Servlet writes the output**
   Your code reads input via the request object and writes HTML/JSON/etc. onto the response object's output stream.

8. **Response travels back**
   The container converts the response object into a real HTTP response, sent back through the web server to the browser. The thread then ends — but the servlet instance stays alive for the next request.

Request & Response, Up Close

These two objects are the only way a servlet talks to the outside world — everything about the call in, and everything about the answer out, lives inside them.

`HttpServletRequest`

Everything the client sent, wrapped in one object.

- `getParameter()` — form fields & query params
- `getHeader()` — HTTP headers
- `getSession()` — the user's session
- `getCookies()` — cookies sent by the browser
- `getMethod()` — GET, POST, PUT...

`HttpServletResponse`

Everything the servlet sends back, before it's turned into bytes on the wire.

- `setStatus()` — HTTP status code (200, 404...)
- `setHeader()` — response headers
- `getWriter()` — write text/HTML output
- `getOutputStream()` — write binary output
- `setContentType()` — e.g. `text/html`, `application/json`

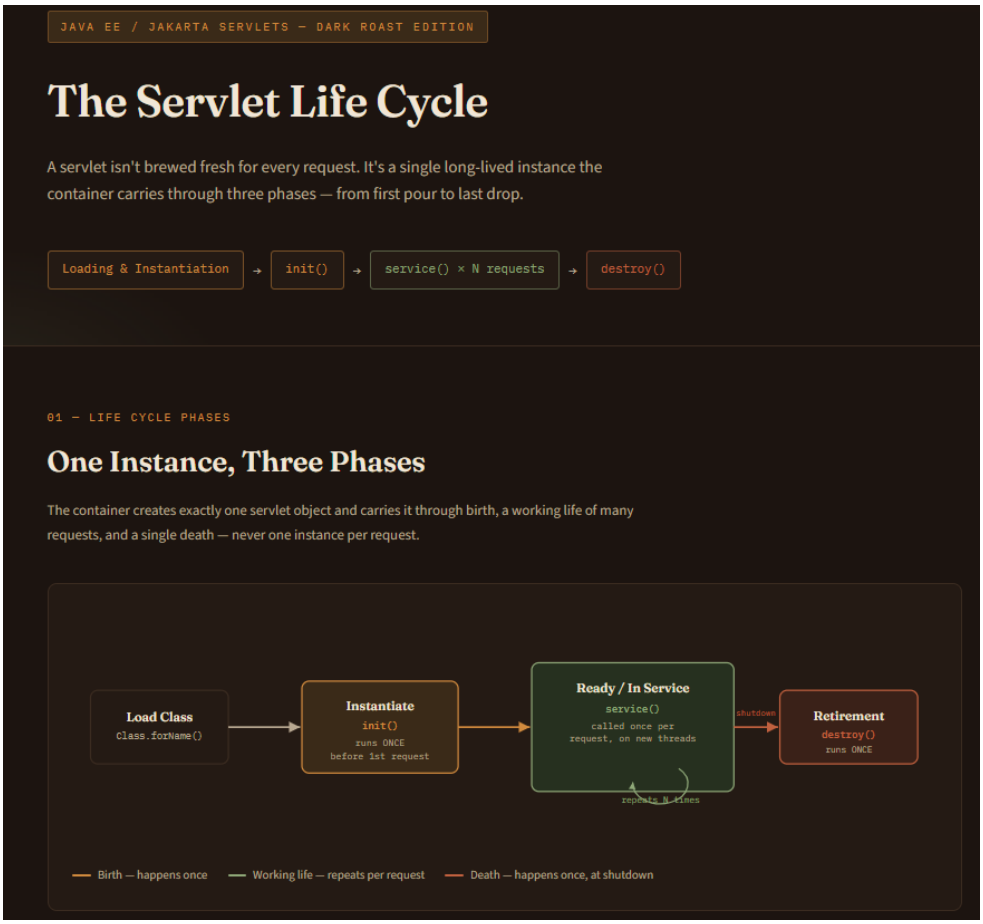
2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of init(), service(), and destroy() methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications



Sequence: Container ↔ Servlet

Read top to bottom as a timeline of calls the container makes on the servlet instance across its whole life.

1. **Container loads the servlet class**
   Happens on server startup, or lazily on the first matching request, depending on `load-on-startup`.

2. **Container instantiates the servlet**
   Exactly one object is created via the no-arg constructor. This same object serves every future request.

3. `init(ServletConfig)` **is called**
   Runs exactly once, before the servlet can accept any request. Used to open resources: DB connection pools, config files, caches.

4. **Request #1 arrives →** `service()`
   Container creates Request/Response objects and calls `service()` on a new thread. It dispatches to `doGet()` / `doPost()`.

5. **Request #2, #3 … #N →** `service()` **again**
   Every new request reuses the *same* servlet instance, each on its own thread, running concurrently.

6. `destroy()` **is called**
   Runs exactly once, when the app is undeployed or the server shuts down — after any in-flight `service()` calls finish. Used to release resources cleanly.

7. **Instance is garbage collected**
   The object is dereferenced and eventually reclaimed by the JVM.

init(), service(), destroy()

Same three methods, three very different jobs and call frequencies.

init()

`called: exactly once`

Prepares the servlet for work. Given a `ServletConfig` with init-parameters.

- Open DB connection pools
- Load configuration / templates
- Initialize caches or counters

service()

`called: once per request`

The workhorse. Reads the HTTP method and routes to the matching handler.

- GET → `doGet()`
- POST → `doPost()`
- Runs concurrently across threads

destroy()

`called: exactly once`

Cleans up before the servlet is retired, letting active requests finish first.

- Close DB connections
- Flush logs / buffers
- Release file handles, threads

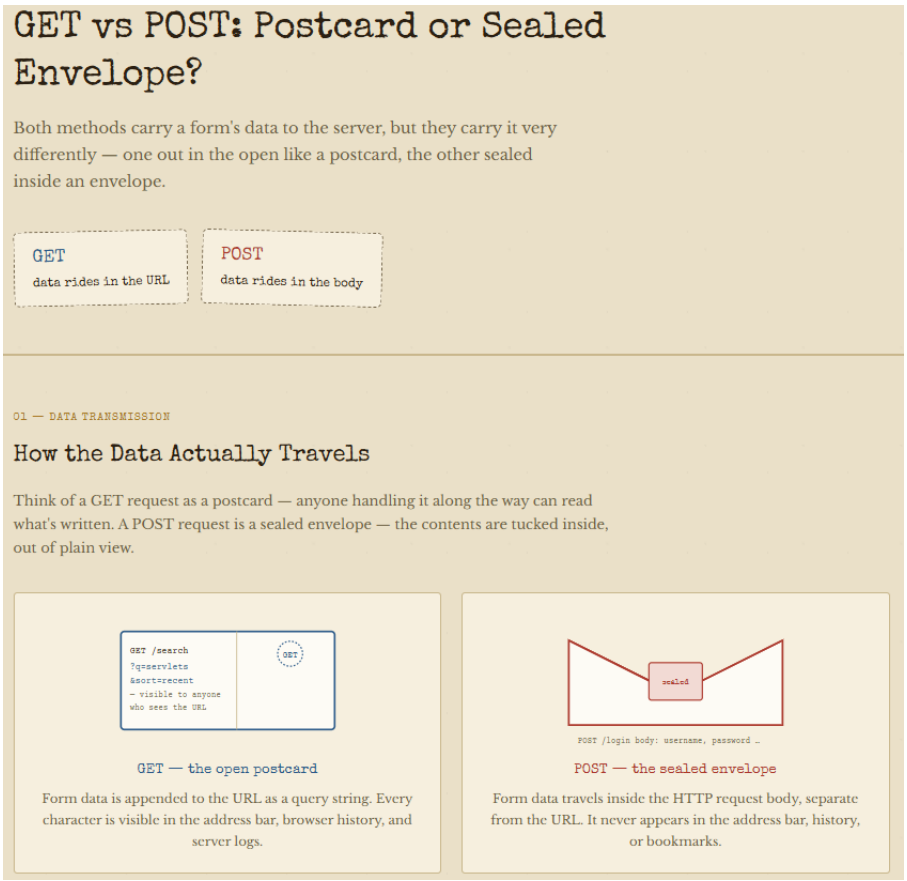
3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations



What Gets Exposed, and Where

Neither method is "secure" on its own — that's what HTTPS is for. But they differ sharply in what gets left lying around afterward.

ASPECT	GET	POST
Browser history	Full data saved with the URL	Only the URL is saved, not the body
Server access logs	Query string is logged in plain text	Body is normally not logged
Bookmarking / sharing	Data travels along if the link is shared	Data is not part of a shareable link
Caching	Can be cached by browsers & proxies	Not cached by default
Suitable for passwords?	No — ends up stored in plain text	Yes, when paired with HTTPS

The Postcard Has a Size Limit

Because GET data lives in the URL, it inherits the URL's practical size ceiling. POST has no such restriction, since its data rides separately in the body.

ASPECT	GET	POST
Practical length limit	~2,000–8,000 chars depending on browser/server	No meaningful limit (server config aside)
File uploads	Not possible	Supported via `multipart/form-data`
Special characters	Must be URL-encoded	Sent as-is in the body
Overflow behavior	Silently truncated or rejected (HTTP 414)	Not affected by URL length at all

Picking the Right Envelope

The choice usually comes down to one question: should this action be shareable, bookmarkable, and repeatable — or private and one-time?

Use GET when…

- **Search & filtering** — `/products?category=books` should be a shareable, bookmarkable link.
- **Pagination** — page numbers belong in the URL so a page can be revisited directly.
- **Read-only requests** — fetching data with no side effects (idempotent by nature).
- **Non-sensitive data** — nothing that would be a problem if it showed up in history or logs.

Use POST when…

- **Login forms** — credentials should never sit in a URL.
- **File uploads** — images, documents, anything beyond plain text fields.
- **State-changing actions** — creating an order, submitting a payment, posting a comment.
- **Large or sensitive payloads** — long text, personal details, anything that shouldn't be cached or bookmarked.

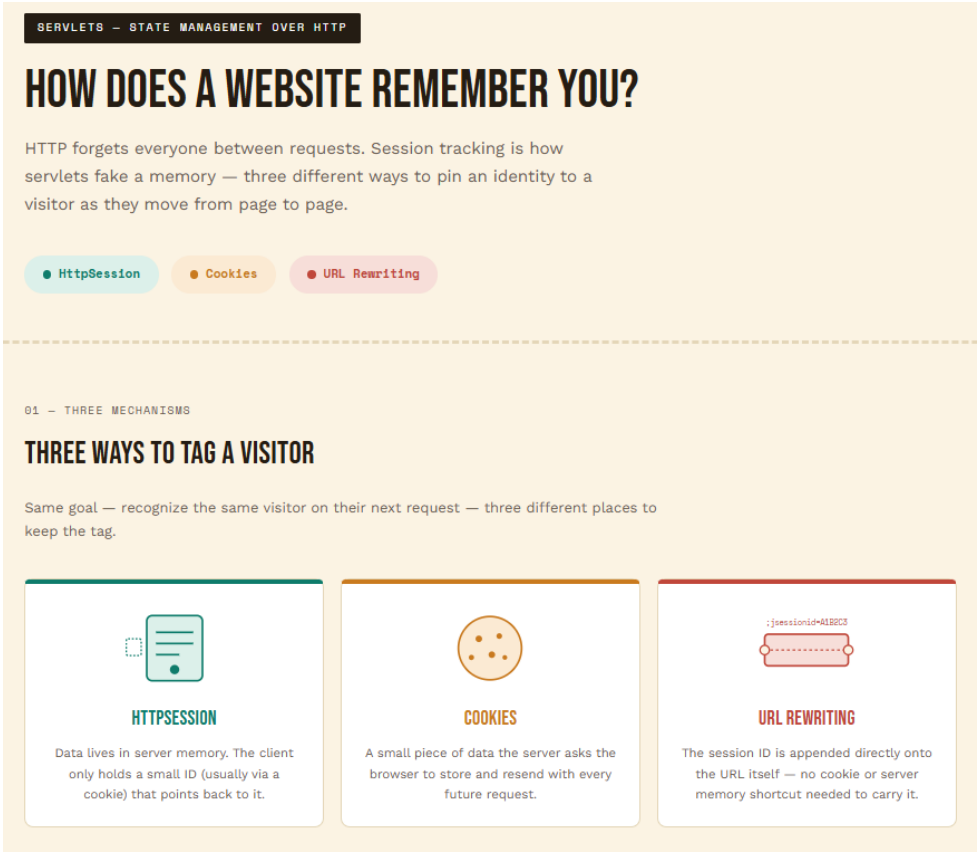
4. Session Tracking Techniques

Question:

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison



WHAT CAN GO WRONG

Each mechanism trades convenience for a different exposure risk.

RISK	HTTPSESSION	COOKIES	URL REWRITING
Exposure point	Only the session ID is exposed (via cookie/URL)	Readable/editable client-side unless flagged	Visible in the address bar, history, referer headers
Session hijacking	Possible if the ID leaks	Mitigated with `HttpOnly`, `Secure` flags	High risk — ID can be copy-pasted or shared by accident
Tampering	Data itself can't be edited by client	Client can edit cookie values directly	ID can be manually altered in the URL
Logged / cached	ID may appear in logs, not the data	Not typically logged by servers	Often logged in server access logs and browser history
Best practice	Pair with HTTPS + short timeout	Always set `HttpOnly` + `Secure`	Use only as a last-resort fallback

WHEN TO REACH FOR EACH

In practice, most apps use HttpSession day to day, with cookies as the delivery mechanism underneath — URL rewriting stays in the back pocket.

HTTPSESSION

- Shopping carts and multi-step checkouts
- Logged-in user state across an app
- Anything needing real server-side objects, not just text
- Default choice for most web applications

COOKIES

- "Remember me" logins that persist across visits
- Storing small preferences (theme, language)
- Carrying the session ID itself, underneath HttpSession
- Lightweight tracking that doesn't need server memory

URL REWRITING

- Fallback when a user has cookies disabled
- Short-lived flows like email confirmation links
- Environments where cookie support can't be assumed
- Rarely the primary mechanism — used as backup only

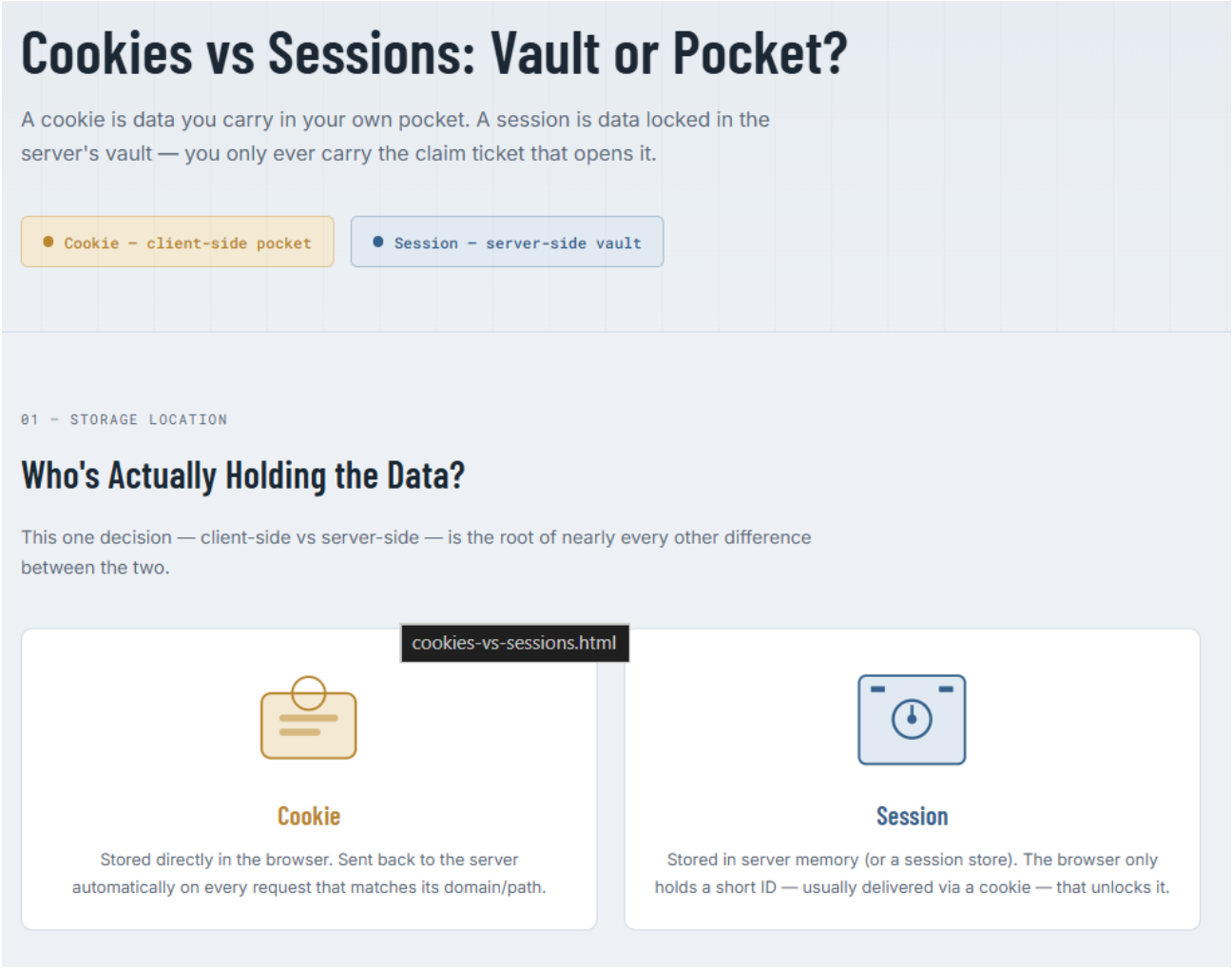
5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
- Security risks
- Performance considerations



Where Each One Is Vulnerable

The vault is harder to break into directly — but the claim ticket that opens it becomes the real target.

RISK	COOKIE	SESSION
Direct tampering	Client can read/edit values unless encrypted	Data itself is never exposed to the client
Theft target	The cookie's actual content	Just the session ID — but that's enough to hijack it
XSS exposure	Mitigated with `HttpOnly`	Same — the session ID cookie needs the same flag
Persistence risk	Can linger on shared/public machines	Expires with server-side timeout, harder to leave behind
Server load from attack	No server-side cost	Flooding with fake sessions can exhaust server memory

Cookie-specific risk

- Never store passwords or sensitive data directly in a cookie
- Always set `Secure` so it's only sent over HTTPS
- Vulnerable to being copied and replayed on another device

Session-specific risk

- Session fixation — an attacker pre-sets a known session ID
- Session hijacking if the ID leaks over an unencrypted connection
- Regenerate the session ID immediately after login

What Each One Costs

Cookies cost you network bandwidth on every request. Sessions cost the server memory for as long as the vault stays open.

ASPECT	COOKIE	SESSION
Per-request cost	Adds to every request's header size	Only a small ID is transmitted; data stays put
Server memory cost	None — server stores nothing	Grows with concurrent users × data per session
Scalability across servers	Naturally stateless — scales easily	Needs sticky sessions or a shared session store
Cleanup	Expires on its own by date, no server work needed	Server must actively expire idle sessions to free memory
Best for large data	Poor — 4KB cap, adds request overhead	Good — no size pressure on the network

Where You'd Actually See Each One

Cookie

e.g. e-commerce site, "remember me"

- Keeping items in a cart for a guest who isn't logged in
- "Remember me" checkbox that logs a user in on return visits
- Storing UI preferences — theme, language, layout
- Tracking ad/analytics IDs across visits

Session

e.g. online banking, checkout flow

- Holding a logged-in banking session with a short, strict timeout
- Multi-step checkout where cart contents must stay accurate
- Tracking a user's progress through a multi-page form
- Anything where the data must never be exposed client-side

Code of Conduct:

I Nisha K (192411231) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Nisha K

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness